An Experimental Study of RPC Inference on Resource-Constrained ARM Single-Board Computers

Hudson Moreira Guimarães do Santos*, Bruno Cavalcante de Souza Sanches*,
Laisa Caroline Costa De Biase*, Roseli de Deus Lopes*, Marcelo Knörich Zuffo*,
Pablo Cesar Calcina Ccori
*University of São Paulo (USP), São Paulo, Brazil

***Abstract*—Partitioning a language model across inexpensive edge devices can extend available memory, but its latency cost must be measured on the target platform. This paper evaluates the RPC backend of `llama.cpp` on Caninos Labrador ARM single-board computers with approximately 2 GiB of memory each. We compare five selected valid executions per condition: notebook-only, an isolated Labrador, and a notebook coordinating two Labrador RPC workers. Qwen2.5-1.5B-Instruct `Q5_K_M` is evaluated with a fixed prompt, 32 generated tokens, deterministic sampling, and a fixed runtime revision. Paired worker logs establish layer assignment and graph execution on both workers. Mean process time increases from 43.86 s on the isolated board to 168.67 s with two RPC workers, a 3.85-fold increase. Prompt processing and generation increase by 1.95 and 1.91 times, respectively; the larger process-time penalty therefore also involves time outside those measured phases. That remainder is not independently attributable to loading or communication. The contribution is an auditable case study of successful distributed execution without latency improvement under a constrained configuration. The findings inform deployment evaluation for local consumer AI while remaining limited to one model, one short workload, and the selected executions.**

***Index Terms*—edge AI, consumer electronics, ARM SBC, local inference, RPC, reproducibility**

I. Introduction

Local language model inference is relevant to consumer devices when connectivity, data locality, and ownership of computing resources motivate execution near the user. Edge-first evaluations show that model selection and hardware affect generation speed, responsiveness, and energy cost [1]. Quantization reduces model storage, while partitioning can combine the memory of several devices [2], [3]. However, intermediate-state transfers and remote coordination can offset the benefit of additional resources. For low-cost ARM boards, proving that distributed inference runs and measuring whether it improves latency are separate requirements.

This work asks: for a fixed quantized instruction model and short deterministic request on approximately 2-GiB ARM boards, does a notebook coordinating two RPC workers complete the request faster than one isolated board? The model also runs on the isolated board, deliberately providing a feasible local baseline. The experiment consequently tests the latency trade-off, not whether partitioning enables a model that exceeds single-board memory. Its scope is a platform case study rather than a new partitioning algorithm or a full cluster-scaling evaluation.

The contributions are: (1) a controlled comparison with five selected valid executions per condition and preserved failure provenance; (2) paired layer and graph-operation evidence establishing participation of both workers in each included RPC request; and (3) a phase-aware analysis distinguishing prompt processing, generation, and an uninstrumented process-time remainder. Together, these show a concrete case in which effective distributed execution does not yield a latency advantage. For a developer evaluating local consumer AI on inexpensive boards, this distinction prevents treating successful offloading alone as evidence of a faster service.

II. Related Work

EdgeShard jointly selects devices and partitions models to optimize latency and throughput in heterogeneous collaborative edge environments [4]. Its algorithmic optimization addresses a broader design problem than the fixed RPC deployment studied here. MDI-LLM partitions transformer models across low-power devices and introduces recurrent pipeline parallelism for generating multiple sequences [3]. Its evaluation examines throughput and memory use, including models that cannot fit on a single device. Our single-request experiment does not test that concurrency mechanism or the memory-capacity benefit.

Jang and Morabito compare language model deployments using quality, generation speed, time to first token, energy, and cost [1]. Their evaluation motivates distinguishing user-visible metrics and deployment conditions, rather than relying on a single aggregate throughput value. We address a narrower hardware and runtime combination and separate process time from the available phase timings; energy, response quality, and first-token latency remain outside our measurements.

The `llama.cpp` RPC backend exposes remote devices for computation offloading [5]; Qwen2.5 supplies the model family used in our workload [2]. These implementation sources identify the evaluated system, while the three research studies above establish its context. Our contribution is a measured baseline with per-worker execution evidence on constrained

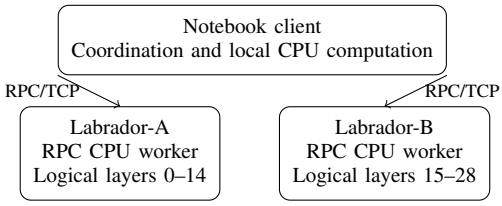


Fig. 1. Evaluated RPC condition. The notebook remains computationally active. Layer 28 denotes the output stage; arrows show client–worker communication, not measured link throughput. Baselines run only on the notebook or an isolated Labrador.

CPU boards. We do not compare numerical performance directly with those studies because their models, hardware, scheduling, and workloads differ.

III. Testbed and Protocol

A. Hardware and software

The testbed contains homogeneous Caninos Labrador 7 boards: ARM64 Cortex-A53, four cores, approximately 1.99 GiB RAM, Debian GNU/Linux 12, and Linux 6.1.76. The inventory records no swap on the boards. They were connected through the `end0` Ethernet interface; the 100 Mb/s/full-duplex figure comes from dated kernel link-up records in the inventory, not from a throughput benchmark. For readability, the notebook is called the client and the two RPC workers are called Labrador-A and Labrador-B. Public raw artifacts retain the original operational identifiers. The notebook executes the client, scheduler, local CPU backend, and any model portion not assigned remotely; it is not a passive controller.

The client log identifies a 12th Gen Intel Core i7-1255U notebook with 31,829 MiB visible memory, Linux `x86_64`, one local CPU backend, and a four-thread `llama.cpp` compute pool. The experiment-time notebook distribution release and RPC worker-thread configuration were not preserved. The client thread count must therefore not be treated as a worker value. Fig. 1 distinguishes the active notebook client from the two remote CPU workers.

The model is Qwen2.5-1.5B-Instruct in `Q5_K_M` GGUF format (SHA-256 `b4666107...896f8c`). The fixed x86 client binary has SHA-256 `13b3c2e9...348d69`; its log reports GNU 15.2. The ARM package was cross-compiled from `llama.cpp` commit `68e79bd8`, with RPC enabled, native CPU detection disabled, and OpenMP, CUDA, Vulkan, Metal, and RDMA disabled. The local compilation variant applies the preserved `-O0` patch to the common and RPC HTTP compilation targets. The package uses GCC 12 and a Debian 12 arm64 sysroot; the ABI audit reports maximum GLIBC 2.34 and GLIBCXX 3.4.30. These are materially different x86 and ARM builds, so the comparison is an end-to-end configuration comparison rather than an isolated architecture effect.

B. Frozen workload

Every selected inference invocation used context 512, four threads, `-ngl 99`, a maximum of 32 output tokens, temperature zero, seed 42, `-ignore-eos`, and no warmup. These invocation settings do not specify the remote worker-thread configuration. The prompt was "Write exactly one short sentence about distributed computing." The runtime reports 38 prompt tokens and, for every included case, 32 actually generated tokens. Ignoring end-of-sequence fixes output length; the prompt is a workload specification, not a sentence-quality test. Each process was newly started; disk caches were not cleared, so the experiment is not a cold-disk benchmark. The process-time metric includes startup, model loading, RPC setup, prompt processing, generation, and residual work not separately instrumented.

C. Validity and selection

The five notebook cases come from `matrix-validation-1`; the five isolated ARM cases come from `matrix-validation-5`; and the five RPC cases combine two valid cases from `matrix-validation-7` with three valid cases from `rpc2-complete-1`. These groups share model hash, client hash, runtime source commit, prompt, sampling parameters, and context. They do not form one consecutive campaign: the launcher and health-check procedure changed before the final three RPC cases, so campaign provenance is retained and the groups are not presented as uninterrupted repetitions.

IV. Worker Participation Evidence

The RPC selection requires more than an open TCP port or a successful client exit. Qwen2.5 has 28 repeating transformer blocks, represented by tensors `blk.0` through `blk.27`. The runtime also exposes logical layer 28 for the output stage; in the selected requests, layers 0–14 were assigned to Labrador-A and layers 15–28 to Labrador-B. Thus layer 28 is not a 29th transformer block: it covers the output-stage tensors such as `output_norm` and `output`. The paired server-log slices contain distinct graph-compute, graph-recompute, and tensor-transfer events for both hosts. For a representative selected request, each host recorded 3 graph-compute events, 30 graph-recompute events, and 33 get-tensor events; set-tensor counts were 345 and 356. Event counts establish activity but do not imply identical computational work. Each request records 38 evaluated prompt tokens, 32 predicted tokens, and normal completion with exit code zero. The earlier instrumented pilot independently established the same participation pattern; it is cited as validation evidence and was not repeated for this paper.

The notebook remains part of the computation, so the comparison is notebook-only versus one ARM board versus notebook-plus-two-workers. Worker events establish participation, not a compute-only utilization percentage or a balanced partition in execution time.

V. Results

A. Latency and phase accounting

Table I shows that RPC-2 takes 3.85 times as long as the isolated Labrador in mean process time. The corresponding

TABLE I
SELECTED VALID EXECUTIONS AND PROCESS-TIME STATISTICS. VALUES ARE RECONSTRUCTED FROM RAW LOGS; SD IS SAMPLE STANDARD DEVIATION.

Condition	n	Mean (s)	SD (s)	Min (s)	Max (s)
Notebook	5	5.69	0.04	5.61	5.71
Labrador	5	43.86	0.43	43.52	44.58
RPC–2	5	168.67	0.16	168.49	168.83

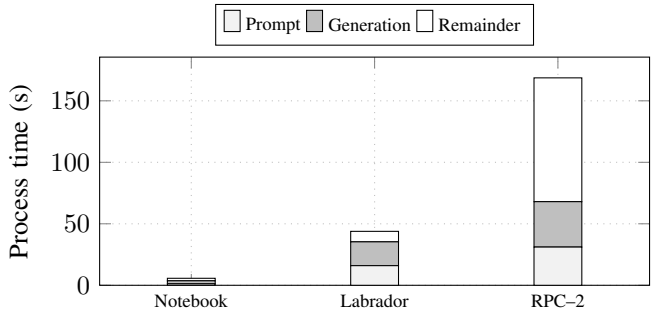


Fig. 2. Mean process time decomposed into runtime prompt and generation timings and their arithmetic remainder. The remainder is not a separately measured loading or communication phase. Total-time variability is reported in Table I.

ratios for prompt processing and generation are 1.95 and 1.91. Thus, the penalty is not confined to startup: both measured inference phases are slower as well. The notebook-only condition provides a reference for local execution on the stronger machine; it is not a matched ARM architecture control.

For each selected execution i, we compute the process-time remainder

$$R_i = T_{\text{process},i} - T_{\text{prompt},i} - T_{\text{generation},i}, \tag{1}$$

after converting phase timings from milliseconds to seconds. Table II and Fig. 2 report the means. RPC-2 has 100.682 s outside the two measured phases, or 59.7% of process time; the isolated board has 8.536 s (19.5%). This accounting localizes the measurement gap but does not identify its cause: R_i can contain process startup, model transfer/loading, coordination, teardown, and other costs outside the reported phases. It is neither time to first token nor a measured network-transfer duration.

The generation rate is the arithmetic mean of the individual `predicted_per_second` fields in the final runtime JSON responses, not 32 divided by mean generation time. The reconstruction CSV retains the latter ratio separately. Low process-time standard deviations describe repeatability within these selected cases; they do not quantify variability across prompts, sessions, or deployment environments.

B. Failure provenance

The fifteen selected cases are valid performance observations, not a reliability sample. The complete inventory contains 55 logged attempts across campaigns and conditions. Table III groups the excluded failures; other normally completed cases outside the fixed selection are retained in the reconstruction CSV.

TABLE II
MEAN TIMINGS AND RUNTIME GENERATION RATES FOR SELECTED VALID CASES. EVERY CASE GENERATES 32 TOKENS. REMAINDER IS COMPUTED PER EXECUTION USING (1).

Condition	Prompt (s)	Gen. (s)	R (s)	Rate (tok/s)
Notebook	1.222	2.426	2.042	12.78
Labrador	15.948	19.377	8.536	1.60
RPC–2	31.076	36.914	100.682	0.84

TABLE III
FAILURE INVENTORY RECONSTRUCTED FROM ALL SCANNED ATTEMPTS; CAMPAIGNS ARE NOT POOLED INTO A SINGLE SUCCESS DENOMINATOR.

Failure class	Count
Connection/startup	16
Preparation/tool/host	10
Loading timeout/kill	1
Incomplete timeout/interrupt	3
Partial generation	2

The five valid cases per condition were selected using explicit selection criteria: valid status, complete phase metrics, actual token count, and worker evidence. They were not five consecutive successes. The failure table distinguishes preparation and host availability, connection/startup, loading timeout, incomplete termination, and partial generation; a zero exit code alone was not sufficient for inclusion. Because these attempts span different campaigns and launcher procedures, no single operational success percentage is reported here.

VI. DISCUSSION AND LIMITATIONS

A. Implications for local consumer AI

For a short request on these boards, distributing computation is feasible but increases process time and reduces the runtime-reported generation rate relative to the isolated board. Since the selected model already fits locally, the measured configuration offers no latency justification for using two remote workers. A device developer should establish the local baseline and distinguish capacity, startup responsiveness, and steady-state generation before selecting distributed execution. This is a deployment lesson from one configuration, not a recommendation against partitioning when a model cannot fit on one device.

The result complements EdgeShard and MDI-LLM rather than contradicting their reported improvements [3], [4]: those systems target optimized allocation or pipelined multi-sequence execution under different hardware and memory constraints. Likewise, an unfavorable process time here does not establish an unfavorable energy, quality, or cost trade-off of the kind considered by edge-first studies [1]. Our measurements address one latency question only.

B. Threats to validity and next measurements

The latency difference is consistent with remote execution, intermediate-state communication, serialization, and coordina-

tion costs, but the dataset does not isolate those mechanisms. In particular, neither the recorded link mode nor the arithmetic remainder establishes that network bandwidth dominates the observed penalty. Different x86/ARM compiler builds and the local `-O0` workaround are part of the evaluated configuration, not controlled independent variables. Detailed RPC logging may also affect timing; its overhead was not measured separately.

The experiment has one model, one short prompt, one quantization, one network segment, and five selected valid cases per condition. Separate campaigns and launcher changes limit a causal interpretation of differences between conditions. Selecting complete runs permits a bounded performance comparison but does not estimate service availability. Loading, time to first token, peak memory, and direct communication cost are absent; before/after memory snapshots cannot establish peak use. Worker-thread settings and the experiment-time notebook distribution are also missing. Response quality, energy, privacy leakage, concurrent requests, and models exceeding single-board memory are untested. No particular consumer application was evaluated.

The next measurements should separate model loading and first-token latency, quantify transfer volume and logging overhead, and vary input/output lengths under a matched software setup. These would extend the workload coverage and test explanations for the observed process-time gap.

C. Evidence preservation and access

Raw logs, per-execution metrics, failure classifications, analysis scripts, and the build patch are publicly available in the project repository.[1] The experimental evidence is pinned to commit `7d94be2` [6]. The selected-case CSV and summary are under `research/data/derived/`; the raw runs are under `research/data/runs/`. Running `analyze_rpc_validation.py` in `research/scripts/` reconstructs the source metrics and failure inventory from the recorded outputs without launching inference or accessing the cluster. The additional phase accounting follows (1). Full model and runtime hashes, compiler options, and a build recipe accompany the records. Original testbed identifiers are retained with the owner's authorization. Public availability enables inspection of the evidence; independent rerunning of the hardware experiment has not been established.

VII. Conclusion

We evaluated RPC inference on approximately 2-GiB ARM boards using a fixed Qwen2.5-1.5B `Q5_K_M` workload. Paired logs establish effective execution on both workers for each of five selected RPC requests. Mean process time rises from 43.86 s on one board to 168.67 s with notebook coordination and two workers; both prompt processing and generation also slow down. The larger process-time ratio exposes an additional measurement gap outside those phases, without identifying a unique bottleneck. The study provides a documented baseline for evaluating distributed local AI on constrained devices and motivates targeted loading and communication measurements. It does not establish a general cluster-scaling limit or measure the benefit of pooling memory for a larger model.

[1] https://github.com/hudsonmoreira-spec/labrador-distributed-llm-benchmarks

Acknowledgment and AI-use disclosure

OpenAI Codex, an AI coding agent, was used to organize logs and manifests, draft and revise portions of the Introduction, Related Work, Testbed and Protocol, Results, Discussion, and Conclusion sections, and prepare or revise the Python analysis and LaTeX asset scripts. All reported values in this manuscript were reconstructed from preserved logs by the repository scripts. The authors remain responsible for verification, technical claims, citations, and the final text. This disclosure identifies the AI system and the affected content in accordance with IEEE guidance on AI-generated text [7].

References

[1] S. Jang and R. Morabito, "Edge-first language model inference: Models, metrics, and tradeoffs," in *2025 IEEE 45th International Conference on Distributed Computing Systems Workshops (ICDCSW)*, 2025, pp. 309–314. [Online]. Available: https://doi.org/10.1109/ICDCSW63273.2025.00058

[2] Qwen, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115v2*, 2025. [Online]. Available: https://arxiv.org/abs/2412.15115v2

[3] D. Macario, H. Seferoglu, and E. Koyuncu, "Model-distributed inference for large language models at the edge," in *2025 IEEE 31st International Symposium on Local and Metropolitan Area Networks (LANMAN)*, 2025, pp. 1–6. [Online]. Available: https://doi.org/10.1109/LANMAN66415.2025.11154542

[4] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, "EdgeShard: Efficient LLM inference via collaborative edge computing," *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 119–13 131, 2025. [Online]. Available: https://doi.org/10.1109/JIOT.2024.3524255

[5] ggml-org, "llama.cpp rpc backend," GitHub repository documentation, 2026, source tree at commit 68e79bd8cd6b7995f8fce8da252b249bbf237e63, accessed 2026-10-04. [Online]. Available: https://github.com/ggml-org/llama.cpp/tree/68e79bd8cd6b7995f8fce8da252b249bbf237e63/tools/rpc

[6] Labrador Distributed LLM Benchmarks contributors, "Labrador distributed LLM benchmarks: Experimental evidence and reconstruction scripts," GitHub repository, 2026, evidence snapshot at commit 7d94be2, accessed 2026-10-16. [Online]. Available: https://github.com/hudsonmoreira-spec/labrador-distributed-llm-benchmarks/tree/7d94be29fe4e8069f2c916cb8cd0350acf338f1e

[7] IEEE, "Guidelines for artificial intelligence (ai)-generated text," 2024, accessed 2026-10-04. [Online]. Available: https://open.ieee.org/author-guidelines-for-artificial-intelligence-ai-generated-text/